

QARM V2

Quantitative Alignment Multi-Modal Recommendation for Reasoning User Sequence Modeling

Tian Xia, Jiaqi Zhang, Yueyang Liu, Hongjian Dou, Tingya Yin, Jiangxia Cao, et al. (28 authors)
Kuaishou Technology, Beijing

arXiv:2602.08559 (Feb 2026) · predecessor QARM: arXiv:2411.11739 (Nov 2024)



How to make LLM item understanding useful inside an industrial ranking model.

The paper in one slide

Problem

LLM item embeddings help a recommender only a little:

- their notion of 'similar' is not the business notion (representation unmatched)
- cached embeddings are frozen, so the ranker cannot learn them (representation unlearning)

Two fixes

GSU (retrieval):

- a reasoning LLM filters the item pairs used for alignment
- one LLM, three segments: contrastive + next-token loss

ESU (ranking):

- 2 × Res-Kmeans + 1 × FSQ
→ semantic IDs, fewer collisions

Evidence

Online A/B, multi-week:

- ads revenue +4.873%
- shopping GMV +1.0 to +5.6%

Offline:

- GAUC +0.05 to +1.10 points
- SID collision 77.92% → 32.39%
- retrieval HR@200 (click) 7.77% → 12.5%

An industrial paper: strong deployment evidence, few ablations, several loose ends to discuss.

Glossary 1 · models and building blocks

term	meaning
GSU	General Search Unit: a cheap first step that picks the K history items most related to the target
ESU	Exact Search Unit: an expensive, trained step that attends over those K items
SIM	Search-based Interest Model (Alibaba, 2020): introduced the GSU $\rightarrow$ ESU split for long histories
SIM-hard / -soft	GSU by same category tag (hard) or by similar learned ID embeddings (soft)
TWIN, TWIN V2	Kuaishou's lifelong models (2023, 2024): the GSU reuses the ESU's attention scores
DIN	Deep Interest Network (Alibaba, 2018): attention over the user's most recent items
target attention	attention where the candidate (target) item is the query and history items are keys
MoE	Mixture of Experts: shared experts with per-task gates, one output per task (CTR, CVR, ...)
Item2Item · Swing	item–item similarity from co-clicks (Swing: users who clicked both items)
User2Item · two-tower	a user encoder and an item encoder whose dot product scores user–item pairs
semantic ID (SID)	a short tuple of discrete codes (c^1, c^2, c^3) for an item, derived from its embedding
TIGER, OneRec	generative recommenders that predict semantic IDs (Google 2023, Kuaishou 2025)

Glossary 2 · quantization, metrics and business terms

term	meaning
Res-Kmeans (RQ)	residual K-means: cluster, subtract the centroid, cluster the remainder again, per level
VQ code (QARM)	an item described by the IDs of its K nearest items in embedding space
FSQ	Finite Scalar Quantization: project to a few dims, round each to fixed levels (no codebook)
collision · EdgeNum	share of items sharing their SID with another item · items per SID on average
CTR · CVR · CTCVR	$P(\text{click} \mid \text{shown}) \cdot P(\text{buy} \mid \text{clicked}) \cdot P(\text{click and buy} \mid \text{shown})$
AUC	probability that a random positive is scored above a random negative (50 = chance)
UAUC · GAUC	AUC computed per user, then averaged (UAUC) or weighted by the user's samples (GAUC)
HR@K	hit rate: the share of cases where a true item is among the top K retrieved
exposure · GMV	how often items are shown · gross merchandise value, the money spent on orders
cold-start item	a new item with (almost) no interactions yet, so its ID embedding is untrained
A/B test	live traffic split: part of the users get the new model, the rest the production model
OCR · ASR · ViT	text read from images · speech-to-text · Vision Transformer image encoder

Setting: Kuaishou, short video and live streaming

400M+

daily active users

10M+

new items uploaded
every day

100,000+

interactions in the history
of each active user

One ranking model serves ads, shopping and live streaming. It predicts CTR, CVR, gifts, follows ...

It should use the whole history, but scoring 100,000 items per candidate in streaming training is not affordable. Every lifelong model therefore first cuts the history down.

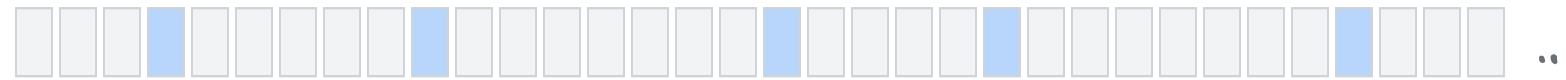
Where LLMs come in

Items are videos, live streams and products with titles, images, OCR and speech. An LLM can read them. The question of both papers: how do we turn that understanding into something the ranker can use?

Lifelong sequences: search first, then attend

The two-stage paradigm of SIM and TWIN

user history (100,000+ items)



GSU · General Search Unit

keep the top-K history items most related to the target (cheap, not trained)

top-K subsequence



ESU · Exact Search Unit

target attention over the K items (expensive, trained end-to-end)

target item

What is 'related'?

SIM-hard: same tag
SIM-soft / TWIN:
similar ID embedding

QARM V2:
GSU → LLM embedding
ESU → semantic IDs

MoE → CTR, CVR, ...

Both stages need an item similarity. QARM V2 supplies it from an LLM instead of from ID embeddings.

Why not just IDs? Three weaknesses of ID embeddings

	ID-based RecSys (weak)	LLM (strong)
Information units	billions of user / item IDs, growing every day	a fixed token vocabulary
Knowledge storage	in ID embeddings; one cross-attention layer	in a deep transformer (40+ layers)
Generalization	needs real-time streaming training; stops → drops	train once; stable long-term inference

Long-tail problem, made concrete

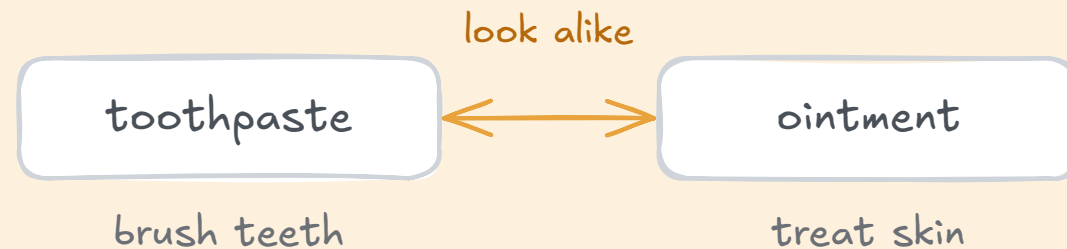
A new item has no interactions yet, so its ID embedding is random: it cannot beat old, hot items.
Once an item is no longer distributed, everything its embedding learned is thrown away ('knowledge isolation').
An LLM reads the title and the images, so it knows what a new item is on day one.

But plugging in LLM embeddings gives little

The common practice: cache one embedding per item, add it as a feature

1 · Representation unmatched

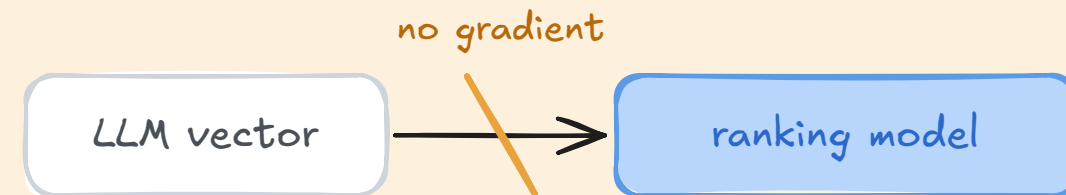
Pre-training learns image-text matching, captions, QA. The ranker needs click and purchase similarity.



Same tube packaging → close embeddings.
Different usage → should be far apart.

2 · Representation unlearning

A cached embedding is a fixed input. No gradient reaches it, so it cannot adapt to the ranking task or to the business as it drifts.

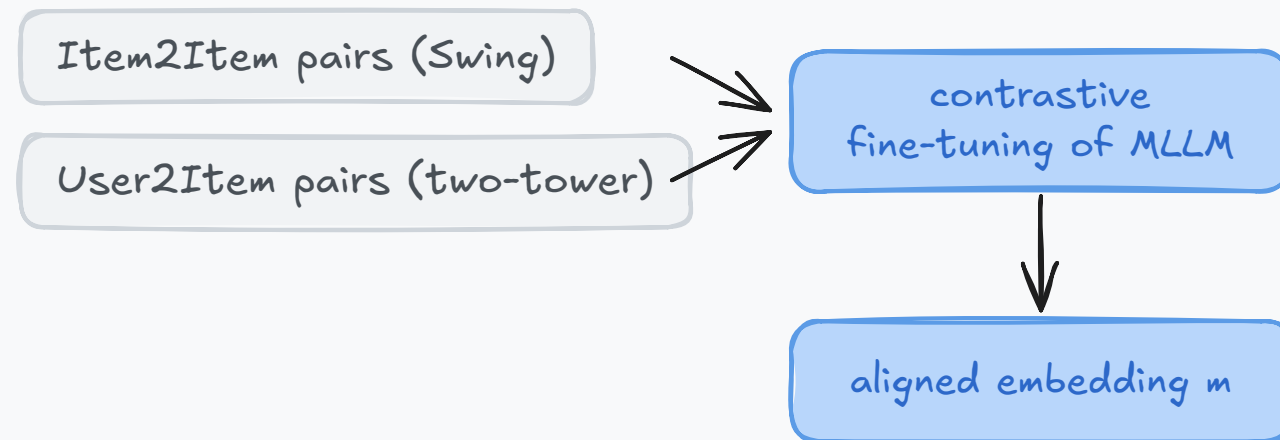


"Our goal is not to recommend look-alike items users have already seen, but to discover new items of interest expanded from their historical behaviors."

The predecessor: QARM (2024)

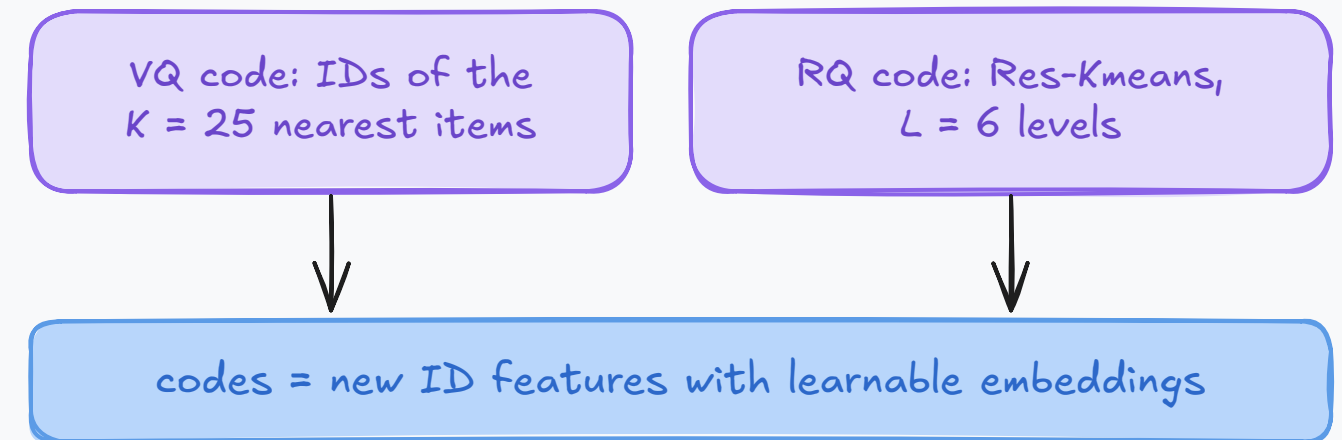
Already attacked both problems; deployed at Kuaishou since March 2024

Item alignment → fixes unmatched



pairs: items clicked together (Swing), and a clicked item + the most similar of the last 50 clicks

Quantitative codes → fixes unlearning



used as item, user-sequence and target-aware cross features in the ranker ($d = 64$)

Key lesson of v1: the aligned embedding as a frozen feature gave +0.02 AUC; as learnable codes, +0.18.

QARM (2024) in numbers: the long tail gains most

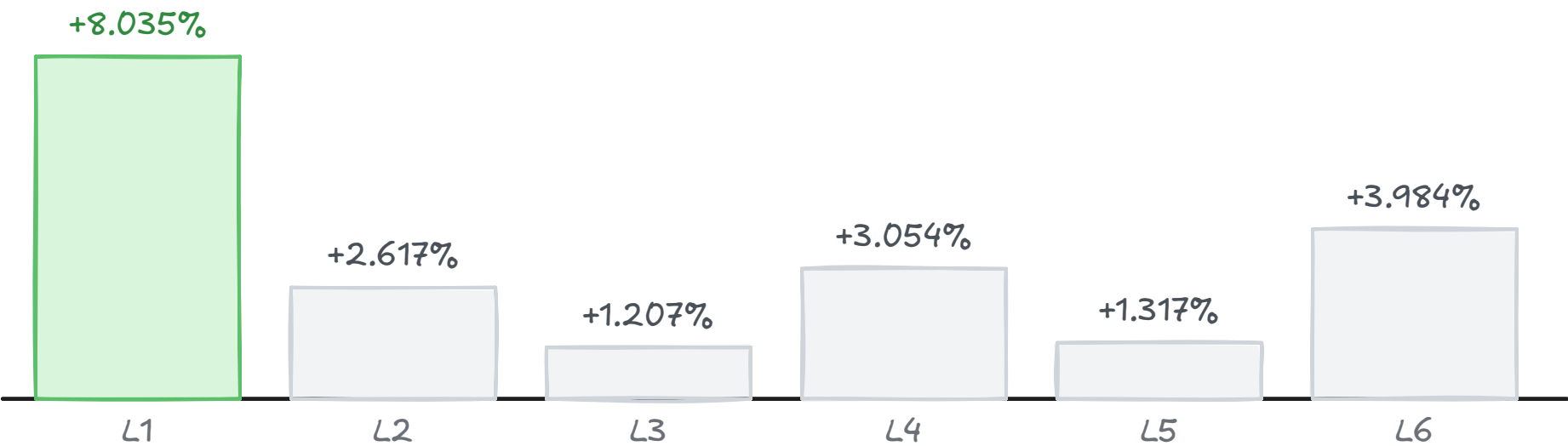
Offline, Advertising CTR (AUC gain over the production model)

variant	AUC
+ aligned embedding (frozen feature)	+0.02
+ VQ code	+0.11
+ RQ code	+0.10
+ VQ & RQ code	+0.18

Online A/B (QARM vs production)

service · metric	gain
Advertising#1 revenue, cold-start items	+9.704%
Advertising#1 revenue, other items	+3.147%
Shopping#1 GMV	+2.296%
Shopping#2 GMV	+1.568%

Shopping#2 GMV by item popularity (L1 = least purchased, L6 = most)



What QARM V2 builds on
the alignment idea, Res-Kmeans codes, the same services. It changes the data, the LLM training and the quantizer.

QARM → QARM V2: what changes

	QARM (2024)	QARM V2 (2026)
Alignment data	raw item pairs from the retrieval models	pairs filtered by a reasoning LLM + generated QA pairs
Embedding LLM	multimodal LLM, contrastive loss	decoder-only LLM, three-segment mask: contrastive + next-token
Semantic IDs	VQ (25 neighbours) + Res-Kmeans (6 levels)	Res-Kmeans (2 levels) + FSQ (1 level)
Use in the ranker	code features (item, user, cross)	embedding → GSU retrieval SIDs → ESU target attention

Part II covers the first two rows (GSU side), Part III the last two (ESU side).

The alignment pairs are noisy

Item pairs exported from retrieval models are the training signal, and some are wrong

(a) Item2Item · Swing (exploitation)

soy sauce



laundry detergent

reasoning LLM:

condiment for cooking vs
cleaning product for clothes

→ "no": reject

co-clicked because both are hot:
exposure bias, no real relation

(b) User2Item · two-tower (exploration)

fashion boots



down jacket

reasoning LLM:

bought together during seasonal
transitions (autumn, winter)

→ "yes": keep

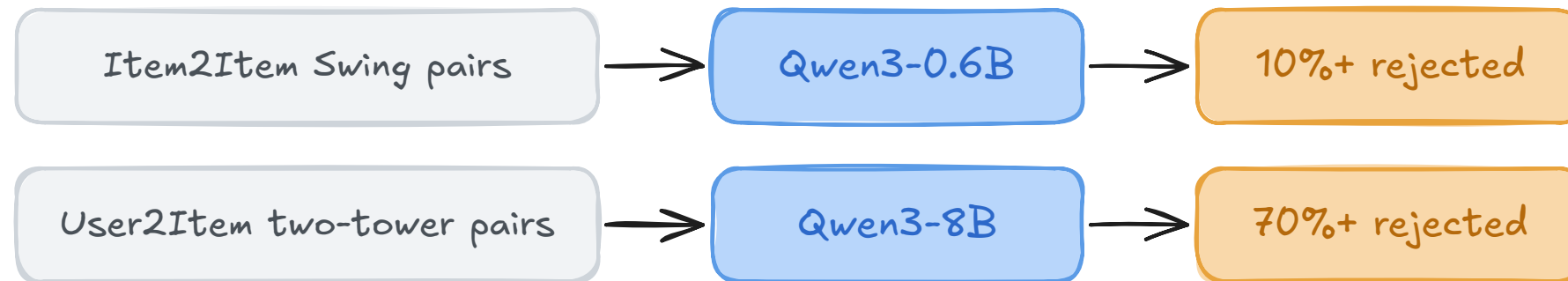
different look, same occasion:
exactly what we want to learn

More examples from the paper: (toothpaste, dental floss) and (fishing rod, parasol) are related; (soy sauce, detergent) is not.

Reasoning data pipeline

A small LLM for easy pairs, a bigger one for hard pairs, a VLM for QA data

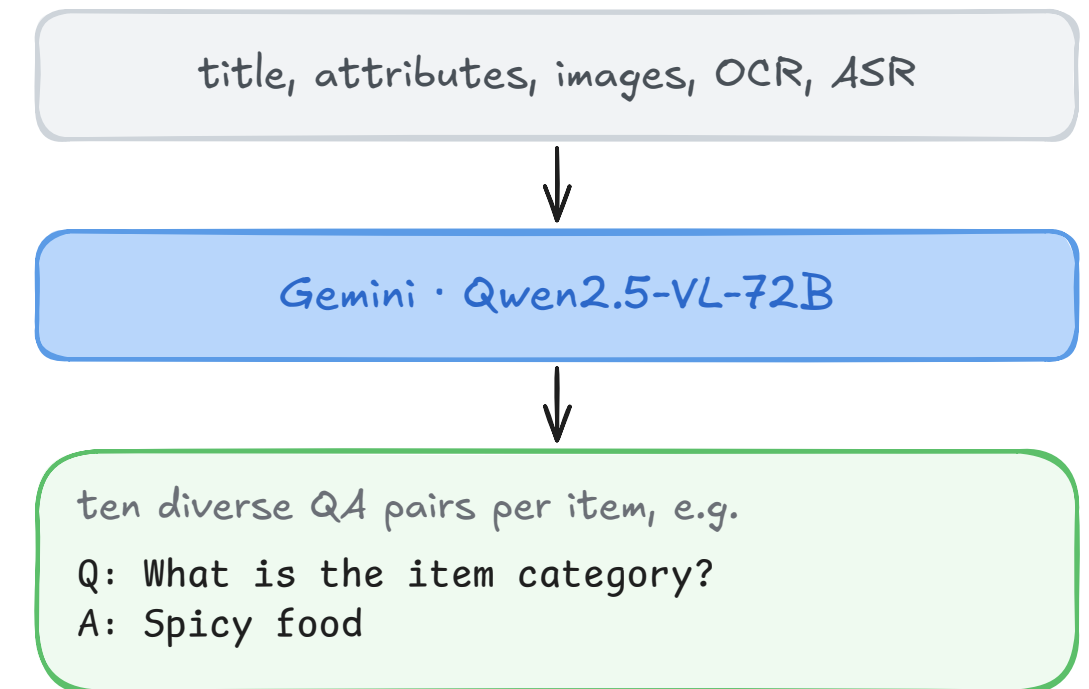
A · Filter item pairs (input: title + attributes only)



prompt (abridged)

Item-1: #Title, #Attributes ... Item-2: #Title, #Attributes ...
 Analyze the potential correlation: semantic (similar concepts or uses) or complementary purchase (bought or used together).
 e.g. (fishing rod, parasol): highly related usage scenarios
 Reply <answer>Yes</answer> or <answer>No</answer>.

B · Generate QA pairs per item

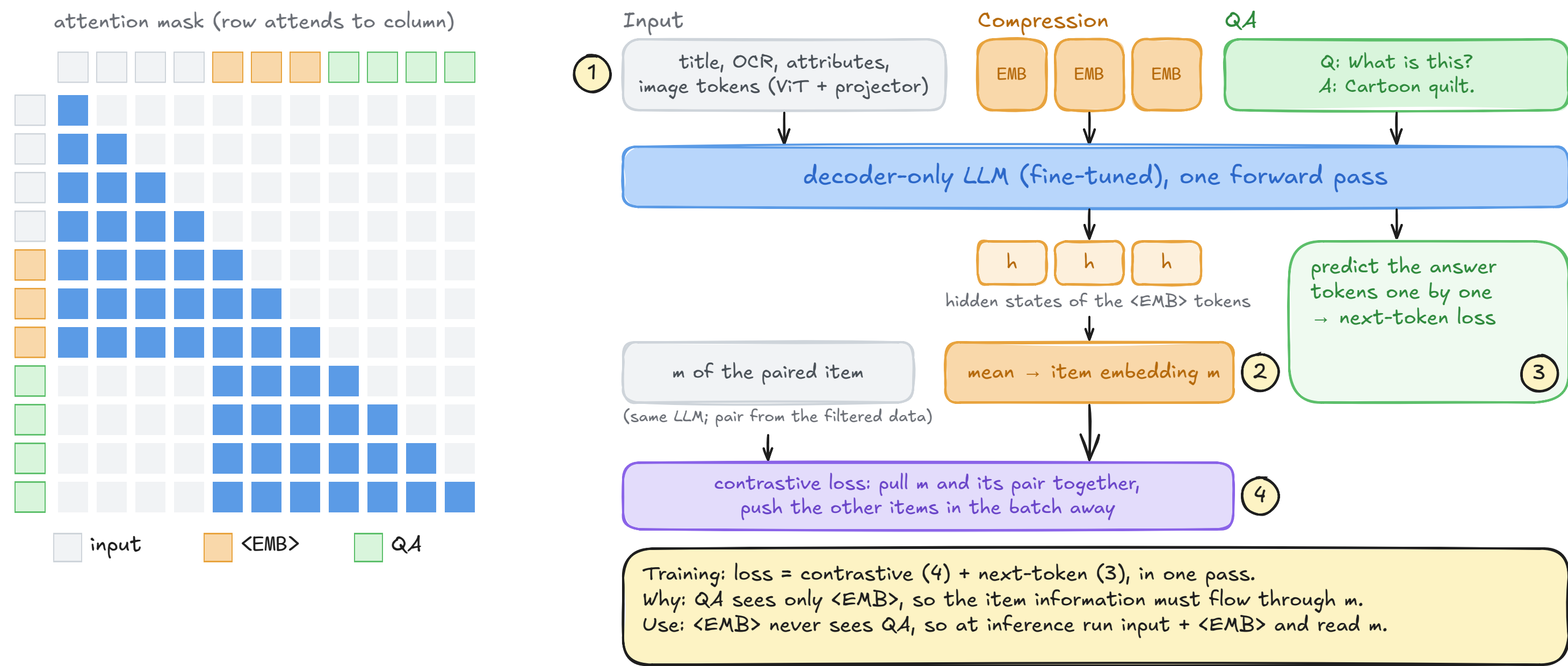


Only questions the model can answer confidently. Used for the generative loss.

Most exploration pairs fail the check: 70%+ of User2Item pairs are rejected (by the larger Qwen3-8B).

Three-segment trick: one LLM, two losses

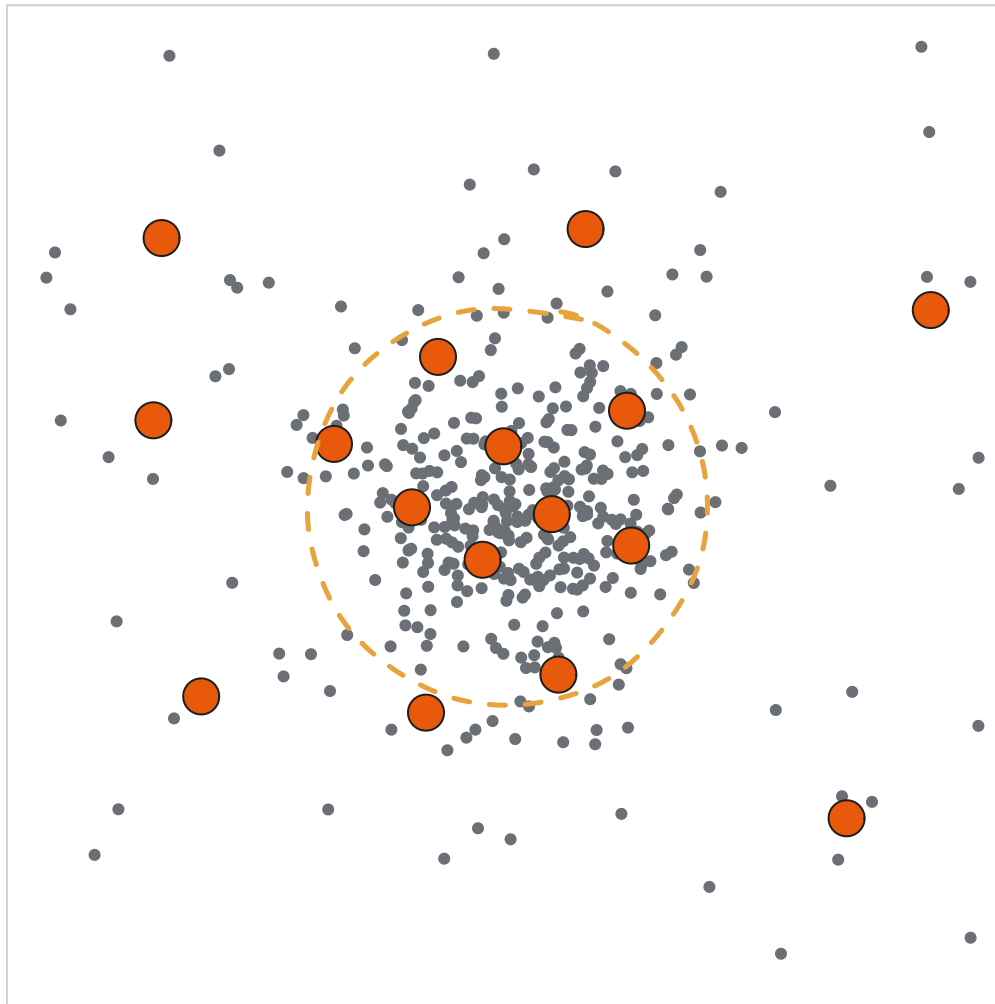
Turn a decoder-only LLM into an embedding model without giving up next-token prediction



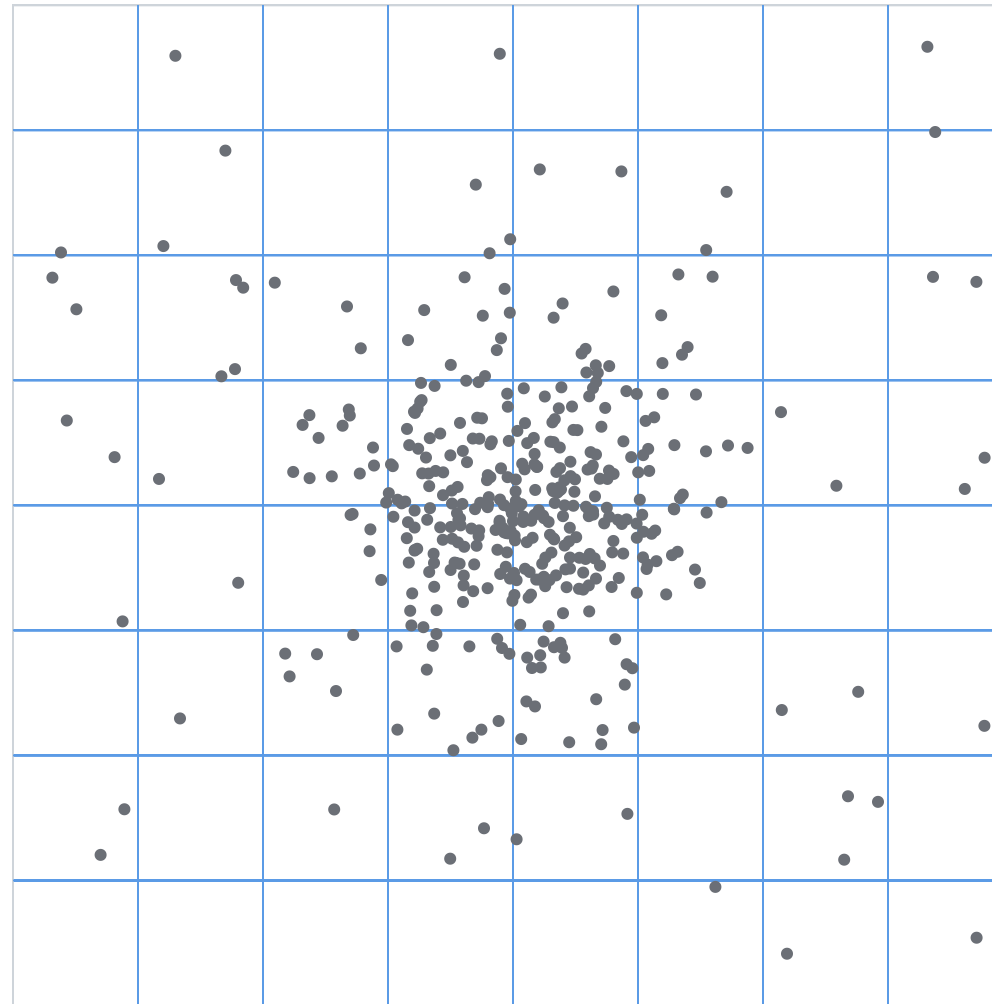
Why semantic IDs collide

A long-tail catalogue has dense and sparse regions; K-means follows the density

(a) K-means: data-dependent



(b) FSQ: data-independent grid



K-means minimises the distance to the nearest centroid only.

Here 9 of 16 centroids sit in the dashed circle, 13% of the area. Tail items far out share a few centroids → same semantic ID.

FSQ rounds each dimension to fixed levels: the grid does not depend on the data, so sparse regions keep their own cells.

In QARM's 3-level Res-Kmeans, more than 30% of semantic IDs map to several items (Shopping).

Res-KmeansFSQ: building a semantic ID in three rounds

Each round describes what the previous rounds missed



Round 1 · coarse

K-means

Offline: K-means on the LLM embeddings of 10M+ items
→ K centroids.

$K = 8,192$ in the method section,
4,096 in the experiments.

c^1 = number of the item's
nearest centroid.

Same c^1 → broadly similar,
roughly the same category.

Round 2 · finer

K-means on the residual

Subtract that centroid. What
remains is the residual: how this
item differs from its cluster's
centre.

Second K-means on all items'
residuals → c^2 = the nearest
centroid there.

(c^1, c^2) = a finer group,
e.g. category + usage.

Round 3 · item detail

FSQ instead of K-means

Subtract the second centroid →
second residual. Then FSQ:

1. multiply by a learned matrix W :
3,000+ numbers → 13
2. squash each into 0–1 (sigmoid),
scale by $L = 2$, round
3. 13 small integers, read together
as one code c^3 , like a
13-digit number

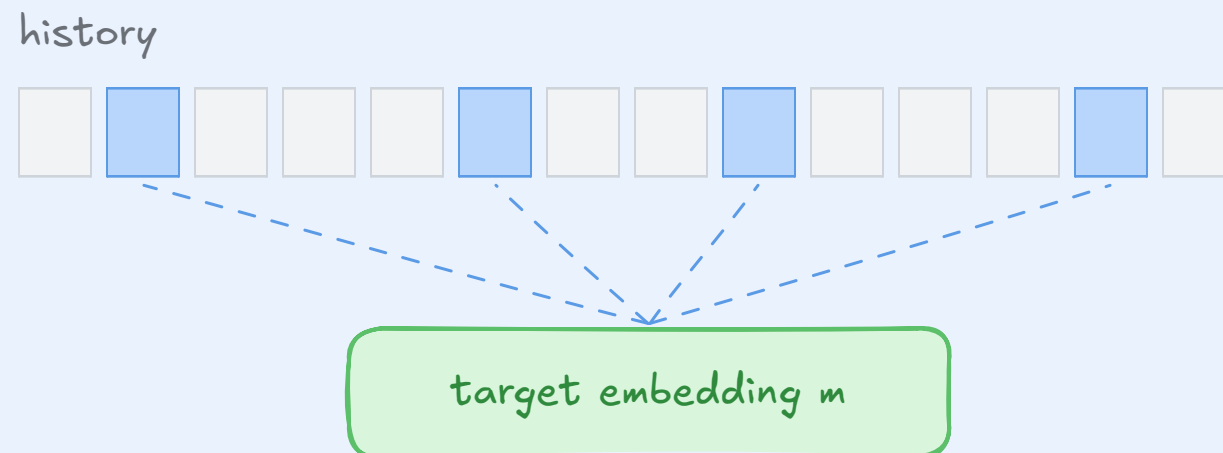
As written, $\text{round}(2 \cdot \sigma)$ gives 3 values per
dim (3^{13} codes); binary dims would give
 $2^{13} = 8,192$. The paper does not say which.

FSQ ignores where items are dense, so it spreads rare items over many different codes. That is the point of round 3.

How the two outputs enter the ranker

GSU: LLM embedding m

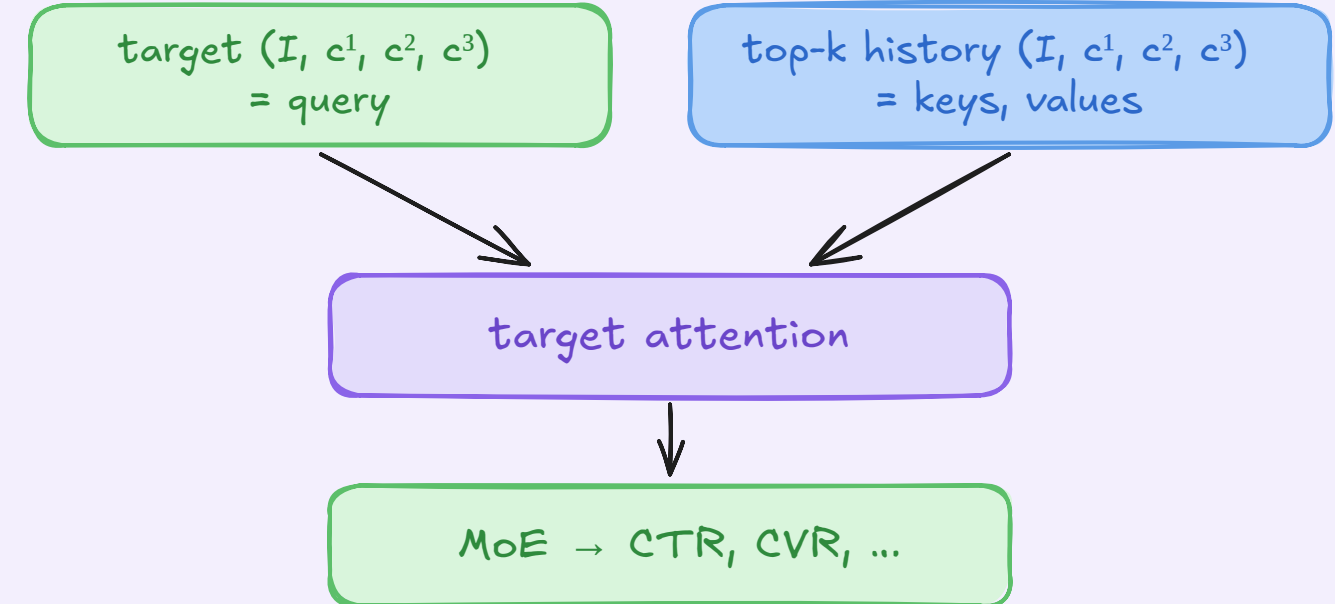
Store m for every item (PCA-reduced).
For a target item, keep the k history items with the highest inner product $\langle m_i, m \rangle$.



Retrieval is now semantic, not ID-based:
new items find related history from day one.

ESU: semantic IDs (c^1, c^2, c^3)

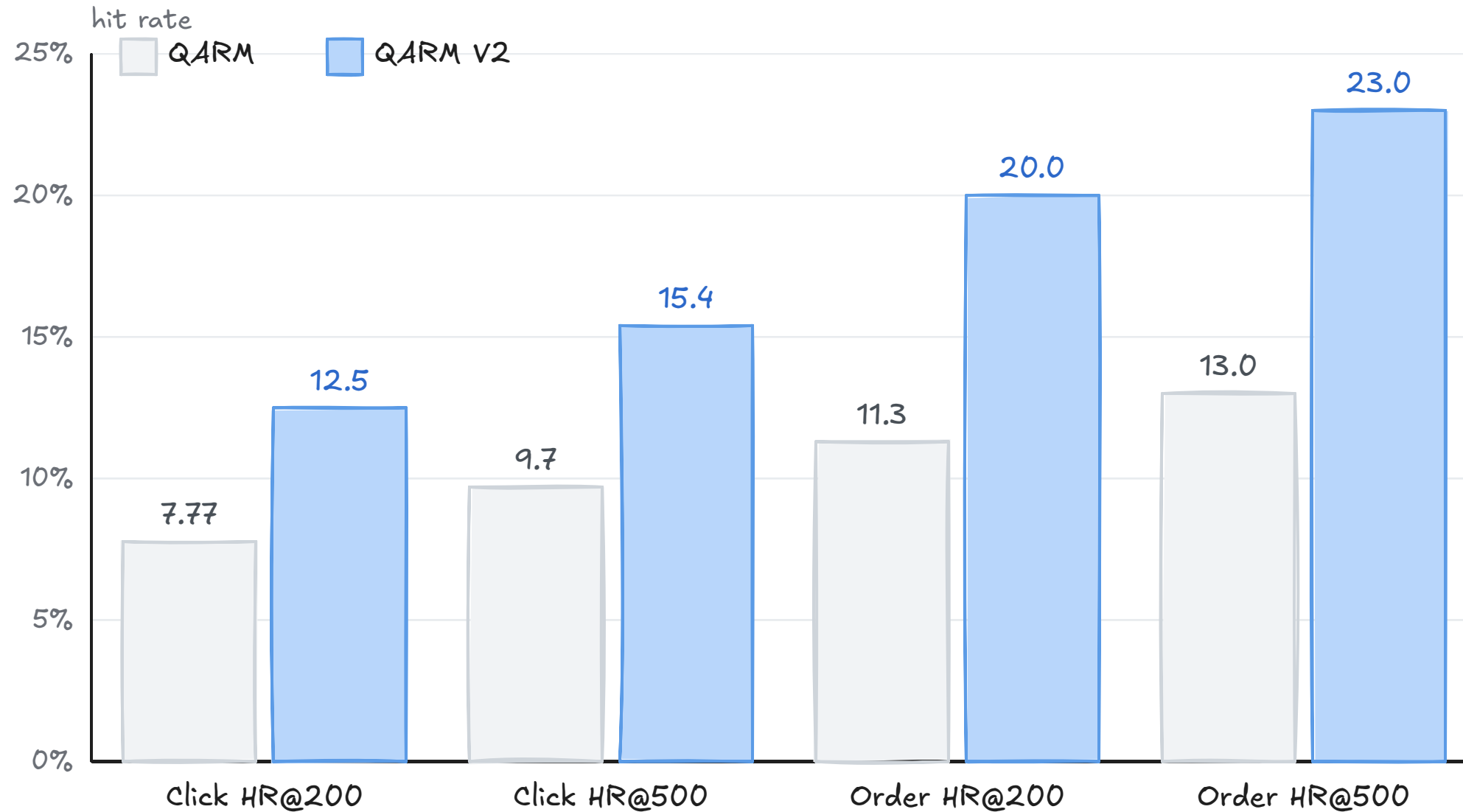
Each item = ItemID + three SIDs, each looked up in its own trainable embedding table.



Multi-task BCE; SID embeddings learned end-to-end.

Did reasoning alignment improve retrieval?

Item-to-item retrieval: 10 recent clicked triggers × 50 candidates; hit = real click or order



Relative gains

click HR@200 +60.9%
click HR@500 +58.8%
order HR@200 +77.0%

The authors read the HR@200 gains as: relevant items are ranked higher, not just found deeper.

Text: order HR@500 rises 'from 20.0% to 23.0%';
the table says 13.0%.

Code collisions: most of the gain comes from better embeddings

all 3 levels × 4096	Collision ↓	EdgeNum ↓	HR@1 ↑	HR@10 ↑
QARM Res-Kmeans	77.92%	129.33	80.3%	97.79%
QARM V2 Res-Kmeans	52.25%	8.41	91.9%	99.75%
QARM V2 Res-KmeansFSQ	32.39%	2.5	95.2%	99.9%

Collision: share of items whose SID is shared with another item. EdgeNum: items returned per SID lookup.
 HR@K: the query item is among the top-K items returned for its own SID.

Same quantizer, new embeddings

EdgeNum 129.33 → 8.41 (15× fewer items per SID)
 from the V2 embeddings alone (new data + training).

Then FSQ on the last level

EdgeNum 8.41 → 2.5, collision 52.25% → 32.39%.
 Per-element MSE by level: $\approx 0.37 / 0.26 / 0.20$.

What the semantic GSU finds that the ID-based GSU does not

history sequence	Click (top 50)	Order (top 30)	Exposure (top 30)
Exclusive rate	63.6%	57.9%	65.1%

Exclusive rate: share of retrieved history items that QARM V2 finds and the baseline GSU (SIM) does not.

ID-based SIM

retrieves 'hard negatives': history items unrelated in category and meaning, e.g. jewellery for a phone-accessory target

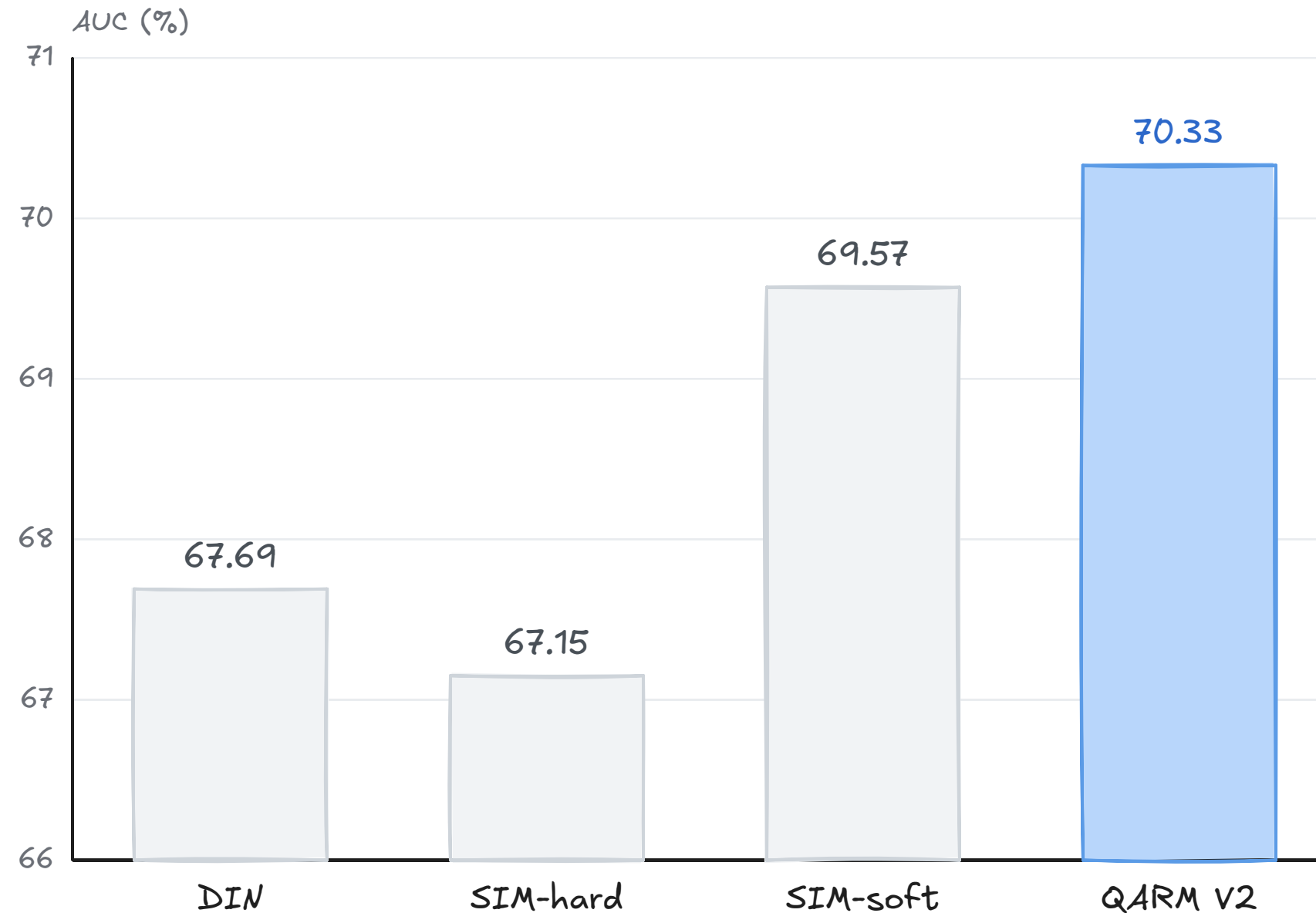
QARM V2

retrieved items stay in the target's category and meaning; often related but visually dissimilar

Also: sequences are not deduplicated. In live streaming the top-100 interactions contain only 23 unique authors on average, and repeats carry signal (watch time, time of day). Deduplication lowered offline AUC.

Public data: Amazon Book

The only public benchmark in the paper



Setup

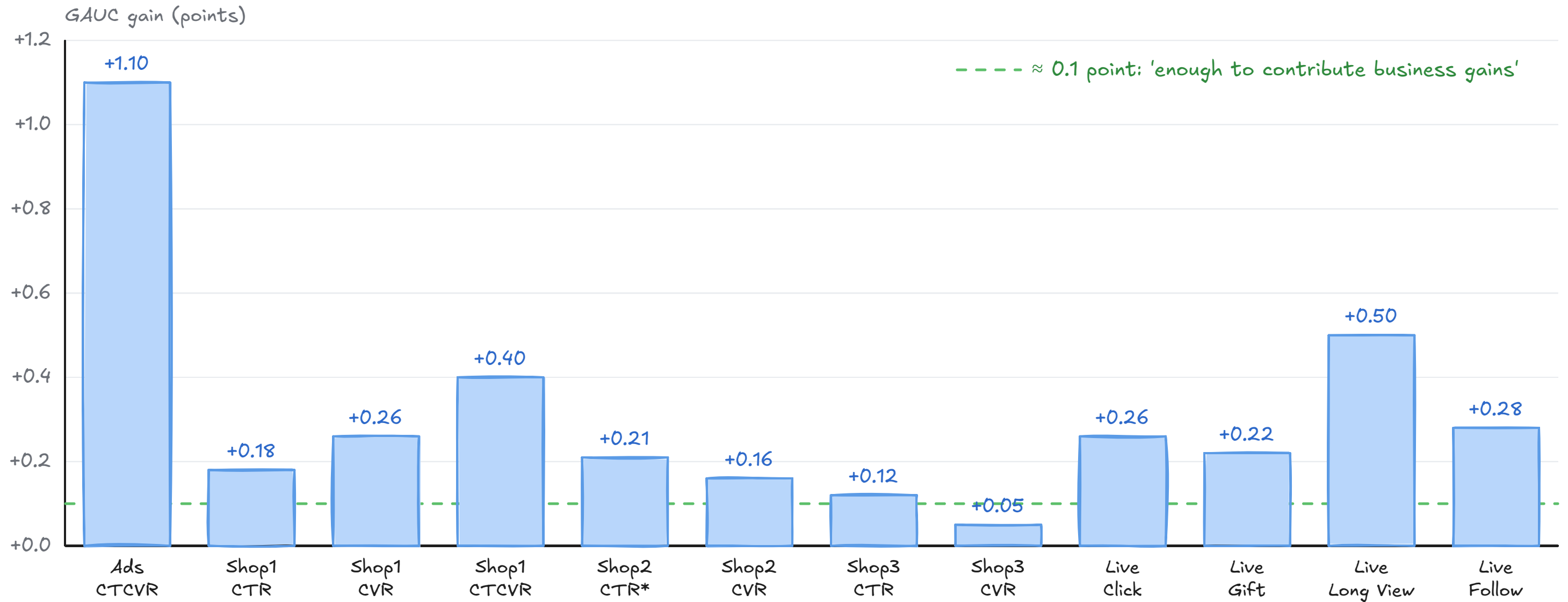
rating $\geq 4 \rightarrow$ positive
users with ≥ 20 interactions
one positive + one negative
per user; 15% of users = test
top-50 items retrieved for ESU

DIN: latest items
SIM-hard: same tag
SIM-soft: ID-embedding top-k

+0.76 AUC over SIM-soft. Note the y-axis starts at 66. No TWIN or semantic-ID baselines (TIGER, OneRec).

Offline at Kuaishou: small but consistent gains

GAUC gain over the production model, in percentage points



Advertising CTCVR: GAUC 63.06 → 64.16, UAUC 62.98 → 63.97. All twelve tasks improve on AUC, UAUC and GAUC. No error bars.

Online A/B: advertising and shopping

Multi-week tests on main traffic

	Exposure	Cost	Revenue
Advertising	+1.321%	+3.942%	+4.873%

	GMV	Order	Exposure
Shopping#1	+3.208%	+1.919%	+0.427%
Shopping#2	+5.612%	+4.834%	+3.739%
Shopping#3	+1.037%	+1.221%	+1.487%

For comparison, QARM (2024) on Shopping: GMV +2.296% (#1) and +1.568% (#2), measured against a different, earlier production model.

Reading the numbers

Revenue grows faster than cost in ads (+4.873% vs +3.942%).

Shopping#2 gains most on all three metrics.

No confidence intervals or traffic sizes are given.

Online A/B: live streaming, cold-start items gain most

core metrics				interaction metrics			
Live-streaming	Click	Watch time	Watch count	Gift count	Like	Comment	Follow
#1 cold-start	+3.231%	+2.961%	+1.609%	+1.807%	—	—	—
#1 others	+0.611%	+0.753%	+0.489%	+2.917%	+3.215%	+1.184%	+1.158%
#2 cold-start	+0.890%	+0.998%	+0.998%	+0.918%	—	—	—
#2 others	+0.304%	+0.482%	+0.434%	+2.917%	+0.464%	+0.826%	+0.418%

Two things to notice

Live-streaming#1 clicks: +3.231% for cold-start streams vs +0.611% for the rest (about 5×): content knowledge helps new items.
Gift count is +2.917% in both 'others' rows: a coincidence, or a copy error in the table?

Reading it critically

Strong

- deployed in ads, shopping and live streaming; multi-week A/B tests
- reusable ideas: an LLM as a data filter, a three-segment mask, a hybrid quantizer
- measures SID collisions directly instead of only end metrics

Weak or unclear

- no ablation on ranking metrics: filter vs three-segment vs FSQ are never separated
- baselines are the production model and, on Amazon, DIN / SIM only (no TWIN, TIGER, OneRec)
- no confidence intervals; some gains are 0.05 points
- inconsistencies: $K = 8192$ vs ' 3×4096 '; FSQ levels; order HR@500 13.0 vs 20.0; collision '> 30%' vs 77.92%; vocabulary < 20k vs $\sim 1e5$
- the filter LLM sees only titles + attributes, and the 10% vs 70% rejection uses different models (0.6B vs 8B)

Takeaways

1 · A frozen LLM embedding is not enough; make it learnable

Turn it into semantic IDs that the ranker learns end-to-end. QARM: frozen feature +0.02 AUC, as codes +0.18.

2 · Align the LLM to business similarity, and clean the pairs first

A reasoning LLM rejects 10%+ of Item2Item and 70%+ of User2Item pairs. Click HR@200: 7.77% → 12.5%.

3 · One LLM can be an embedding model and a generator at once

Three-segment mask: QA sees only <EMB>, so the item embedding m must carry the item's information.

4 · K-means for coarse levels, FSQ for the last level

Collision 77.92% → 32.39%, items per SID 129 → 2.5; but better embeddings alone already gave 8.4 items per SID.

5 · The gains are largest where IDs are weakest

Cold-start live streams: click +3.231% vs +0.611% for the rest. Ads revenue +4.873%, shopping GMV up to +5.612%.

Open question: which of the four changes buys the online gains? The paper has no ablation on ranking metrics.